

Preparación de datos y análisis exploratorio

Introducción a pandas, NumPy y Matplotlib con datos reales

Curso de Inteligencia Artificial

Introducción a Machine Learning

Propósito de la sesión

Al finalizar, podrán:

- 1 Leer un archivo Excel e inspeccionar estructura, tipos y calidad de datos con pandas.
- 2 Resumir distribuciones y relaciones con pandas y NumPy.
- 3 Crear gráficos exploratorios con Matplotlib e interpretar lo que muestran.
- 4 Proponer transformaciones justificadas antes de entrenar un modelo.

Pregunta guía: ¿qué debemos conocer del conjunto de datos antes de predecir ingresos?

El conjunto compartido: una observación por fila

Columna	Tipo observado	Papel posible
id	entero	identificador; no predictor
edad	entero	predictor numérico
nivel_edu	texto	predictor categórico
ingreso_mensual	entero	objetivo numérico posible
sex	texto	variable categórica para auditoría
anios_esc	entero	predictor numérico

Archivo data.xlsx:

10.280 filas de datos, una hoja denominada Sheet1. La unidad monetaria no figura en el archivo: describir importes sin asignar moneda.

Entorno y carga de datos

```
# En una terminal, si faltan las dependencias:  
# python -m pip install pandas numpy matplotlib openpyxl  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
  
ruta = "data.xlsx" # colocar el archivo junto al script  
_df = pd.read_excel(ruta, sheet_name="Sheet1")  
df = _df.copy() # preservar la tabla leida  
print(df.shape) # (10280, 6)  
print(df.head())
```

¿Por qué openpyxl?

Es el motor habitual de lectura de archivos .xlsx para pandas; no es necesario importarlo en el script.

Primer diagnóstico: estructura y tipos

```
print(df.columns.tolist())  
df.info()  
print(df.dtypes)  
print(df.sample(5, random_state=42))
```

- edad, ingreso_mensual y anios_esc son enteros.
- nivel_edu y sex se leen como texto ("object").
- Una muestra aleatoria ayuda a detectar formatos extraños fuera de las primeras filas.

Calidad: ausencias y duplicados

```
print(df.isna().sum())  
print("Filas duplicadas:", df.duplicated().sum())  
print("ID repetidos:", df["id"].duplicated().sum())  
print(df.nunique())
```

Hallazgo en data.xlsx

Cero valores ausentes en las seis columnas, cero filas duplicadas y cero identificadores repetidos.

No imputar por rutina cuando no hay ausencias; conservar los controles en el flujo de análisis.

Categorías: encontrar inconsistencias

```
print(df["nivel_edu"].value_counts(dropna=False))  
print(df["sex"].value_counts(dropna=False))
```

Nivel educativo literal	Registros
Secundaria completa	4.151
Medio superior y superior	3.335
Prrimaria completa	1.823
Primaria incompleta	971

“Prrimaria” es una errata de etiqueta.

Normalizar sin alterar el archivo original

```
limpio = df.copy()
limpio["nivel_edu"] = (
    limpio["nivel_edu"].str.strip()
    .replace({"Prrimaria completa": "Primaria completa"})
)
print(limpio["nivel_edu"].value_counts())
```

- Guardar una copia permite comparar datos originales y preparados.
- La corrección cambia la etiqueta de 1.823 registros; no crea ni elimina filas.
- Si aparecen etiquetas nuevas, revisar su significado antes de fusionarlas.

Describir variables numéricas

```
cols = ["edad", "ingreso_mensual", "anios_esc"]  
print(limpio[cols].describe(percentiles=[.25, .5, .75, .95, .99]))  
print(limpio[cols].median())
```

Variable	Mínimo	Mediana	Máximo
Edad	12	39	98
Ingreso mensual	1.000	6.000	81.000
Años de escolaridad	0	9	22

Media del ingreso: 7.362,39; la diferencia con la

mediana invita a examinar asimetría y valores altos.

NumPy: percentiles y reglas exploratorias

```
y = limpio["ingreso_mensual"].to_numpy()
p25, p50, p75, p99 = np.percentile(y, [25, 50, 75, 99])
iqr = p75 - p25
umbral = p75 + 1.5 * iqr
print(p25, p50, p75, p99, umbral)
print("Casos sobre umbral:", int((y > umbral).sum()))
```

En este archivo: $Q_1 = 3.870$, mediana = 6.000, $Q_3 = 9.000$, $P_{99} = 30.000$ y umbral IQR superior = 16.695.

Interpretación

Un caso señalado por IQR no es automáticamente incorrecto. Verificar unidades, procedencia y propósito antes de decidir.

Matplotlib: distribución del ingreso

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(limpio["ingreso_mensual"], bins=40,
        color="#2868a0", edgecolor="white")
ax.axvline(limpio["ingreso_mensual"].median(),
           color="darkorange", label="Mediana")
ax.set(xlabel="Ingreso mensual (unidad no indicada)",
       ylabel="Frecuencia", title="Distribucion del ingreso")
ax.legend(); fig.tight_layout(); plt.show()
```

Preguntas: ¿dónde se concentra la población?, ¿qué hace la cola derecha con la escala?

Matplotlib: detectar casos extremos

```
fig, ax = plt.subplots(figsize=(7, 3))
ax.boxplot(limpio["ingreso_mensual"], vert=False)
ax.set(xlabel="Ingreso mensual", title="Diagrama de caja")
fig.tight_layout(); plt.show()
```

- La caja representa el rango intercuartílico; la línea interior, la mediana.
- Los puntos fuera de los bigotes son candidatos a revisión, no filas para borrar automáticamente.
- Otra vista útil: histograma de `np.log1p(ingreso_mensual)`; conservar la interpretación en escala original.

Relación entre escolaridad e ingreso

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(limpio["anios_esc"],
           limpio["ingreso_mensual"], alpha=.12, s=10)
ax.set(xlabel="Años de escolaridad", ylabel="Ingreso mensual",
       title="Escolaridad e ingreso")
fig.tight_layout(); plt.show()
print(limpio[["anios_esc", "ingreso_mensual"]].corr())
```

Correlación de Pearson observada: aproximadamente 0,429. Describe asociación lineal; no demuestra que años adicionales causen el cambio del ingreso.

Comparar grupos sin perder el contexto

```
resumen = (limpio.groupby("nivel_edu")["ingreso_mensual"]
           .agg(conteo="size", mediana="median")
           .sort_values("mediana"))
print(resumen)
ax = resumen["mediana"].plot.barh(figsize=(8, 4))
ax.set(xlabel="Mediana del ingreso mensual",
       ylabel="Nivel educativo")
plt.tight_layout(); plt.show()
```

Medianas observadas: primaria incompleta 4.300; primaria completa 5.160; secundaria completa 5.418; medio superior y superior 9.000. Son descripciones de este archivo, sin ajuste por edad u otras variables.

Revisar coherencia y decisiones de limpieza

```
print(limpio.query("edad < 18").shape[0])  
print(limpio.query("anios_esc == 0").shape[0])  
print(limpio.query("ingreso_mensual > 30000").shape[0])
```

- Hay edades de 12 a 98 años y 243 registros con cero años de escolaridad.
- `edad < 18` puede ser válida o incompatible con la población objetivo; revisar la documentación del estudio.
- Un ingreso de 81.000 requiere contraste con la fuente, pero puede ser real.
- Definir cualquier filtro *antes* de ver qué mejora un modelo y registrar cuántas filas afecta.

¿Qué preparación requiere un modelo de ingresos?

- 1 Establecer población, unidad monetaria, fecha y significado de cada columna.
- 2 Elegir $y = \text{ingreso_mensual}$ y predictores disponibles al predecir; excluir `id`.
- 3 Corregir la errata de `nivel_edu`; decidir tratamiento de `sex` según el objetivo y la auditoría de resultados.
- 4 Separar entrenamiento y prueba; ajustar codificación y cualquier transformación aprendida solo en entrenamiento.
- 5 Evaluar errores por grupos y magnitudes de ingreso, no solo un promedio global.

Puente hacia el modelado: evitar filtraciones

```
from sklearn.model_selection import train_test_split
X = limpio[["edad", "nivel_edu", "anios_esc"]]
y = limpio["ingreso_mensual"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Luego: ajustar codificacion/imputacion al conjunto de
# entrenamiento y aplicar las mismas reglas a prueba.
```

scikit-learn aparece aquí solo como anticipo del siguiente tema. Si se usa, instalarlo aparte. Las estadísticas exploratorias generales sirven para entender el archivo; las decisiones de ajuste del modelo requieren aislamiento de prueba.

Actividad guiada: bitácora de preparación

En parejas, con `data.xlsx`:

- 1 Informe forma, tipos, ausencias, duplicados y categorías.
- 2 Haga histograma de ingreso, caja de ingreso y dispersión escolaridad–ingreso.
- 3 Justifique cómo corregiría la errata, trataría posibles extremos y elegiría predictores.
- 4 Escriba dos conclusiones respaldadas por cifras y una pregunta que los datos no permitan contestar.

Producto: cuaderno o script, tres gráficos y una página breve de decisiones. No se requiere entrenar un modelo.

Resumen y documentación

- pandas: cargar, inspeccionar, agrupar y limpiar tablas.
- NumPy: arreglos y cálculos de percentiles.
- Matplotlib: observar distribuciones, extremos y relaciones.
- Preparar datos exige decisiones documentadas y respeto por la partición de prueba.

Material basado en el capítulo 8 de *Grokking AI Algorithms* proporcionado para el curso y en el archivo `data.xlsx` compartido para esta sesión. Documentación: pandas.pydata.org/docs/, numpy.org/doc/, matplotlib.org/stable/.